organization grows. With this safety net, ACME's engineers can refine schemas, modify relationships, or add new guardrails, confident that any unintended effects will be caught before reaching production.

10.4 Pulling it together

Authorization policies are often underestimated. On the surface, they seem minor—just a few lines of Cedar—but they carry enormous weight. A misplaced condition or an untested assumption can either expose sensitive data or grind a business process to a halt.

In this chapter, we've seen how ACME manages that risk by integrating three disciplines:

- Design principles (clarity, least privilege, separation of logic and data, fine-grained actions, stability across change) ensure that policies reflect the correct intent.
- Policy as code makes policies manageable: stored in repositories, reviewed and versioned like any other artifact, deployed into runtime stores with the same rigor as other software.
- Testing and analysis offer reassurance: confirming expected scenarios, preventing regressions, and thoroughly inspecting for structural conflicts or gaps.

Together, these practices remove authorization from fragile, scattered code and place it in a unified, reliable record system. Policies become external, transparent, and trustworthy. For ACME, the benefit is confidence: the identity team knows that when a request is allowed or denied, it is based on a clear, tested decision aligned with business rules.

Designing sound policies, managing them as code, and testing them effectively are the cornerstones of dynamic authorization. With these practices in place, policies become dependable building blocks. But policies don't exist in isolation — they protect real services. Nowhere is this more apparent than in APIs, which have become the front door to most modern systems. In the next chapter, we'll see how ACME applies Cedar to